Paraphrase-Resistant Detection of AI-Driven Code Rewrites: A Falsifiable Harness Applied to the chardet v6 to v7 Relicensing Dispute

Werner Kasselman
Verivus OSS
werner@verivus.com

Abstract

In March 2026 the maintainer of the Python `chardet` library released version 7.0.0 as an MIT-licensed "ground-up rewrite," produced with an LLM-driven coding agent, of a codebase that had carried an LGPL licence since 2008. The relicensing drew objection immediately, with critics arguing that any rewrite informed by an LLM that had seen the original cannot be a clean-room reimplementation and therefore cannot strip the original's copyleft (an argument that is going to come up again, repeatedly, as more production codebases get the same treatment). I am not trying to adjudicate the chardet dispute, and this paper does not take a position on whether v7's relicensing is valid; what I am trying to do is move the conversation from assertion to evidence, by shipping a detection harness that produces eight structural and behavioural, falsifiable, reproducible signals (AUX1 file-hash baseline plus C06a, C06a′, C06b, C06c, C06d, C06e, C06f) and runs them across *three* calibration pairs rather than the v6/v7 pair alone: the v6/v7 LGPL-to-MIT pair under dispute, v5/v6 as a conventional same-project release-to-release evolution baseline, and v6/charset_normalizer as an independent same-domain reimplementation baseline. The headline finding is that the structural-similarity signals the v1 framing leaned on do not separate v6/v7 from those calibration baselines: C06a call-graph similarity is 0.881 on v6/v7 but 0.930 on v5/v6 and 0.922 on v6/charset_normalizer (an independent same-domain library scores *higher* on this signal than the disputed pair), and the C06c control-flow cosine similarly saturates (0.984 / 0.995 / 0.999). The new C06f per-function shape similarity, reported with bootstrap CIs, does discriminate: v6/v7 sits at 0.913 [0.89, 0.94] over only $31/177 = 17.5\%$ of v6 functions matched, between conventional evolution (0.982 [0.97, 0.99] at 64.0% match) and independent same-domain rewrite (0.796 [0.75, 0.84] at $77/177 = 43.5\%$ of v6 functions / $77/148 = 52.0\%$ of `charset_normalizer` functions); the v6/v7 match *rate* is the lowest of the three, which is the signature this paper labels "AI-driven rewrite." On the v1 random-byte workload C06e behavioural agreement is 0.000 for v6/v7 (matching the 0.000 on v6/charset_normalizer) versus 0.968 for v5/v6, cleanly separating routine evolution from any non-evolutionary pair. On realistic input (HTML, multilingual, email, RFC bodies) the new normalised family-agreement rate is 0.625 for v6/v7, 0.688 for v5/v6, and 0.594 for v6/charset_normalizer, so behavioural calibration separates evolution from non-evolution but does not separate AI rewrite from independent rewrite inside the non-evolutionary band. C06b third-party-imports Jaccard, recomputed under the R1–R5 audit rules, rises to 0.667 on v6/v7 (above 0.250 on v5/v6 and 0.000 on v6/charset_normalizer), so the v1 "external boundary is genuinely different" framing inverts. The combined picture remains legally ambiguous, but for a different reason than the v1 draft argued: structural signals do not discriminate derivation from convergence, behavioural calibration separates evolution from non-evolution but does not separate derivative-rewrite from independent-rewrite within the non-evolutionary band, and the v6/v7 pair sits in a strange middle ground that matches

neither a normal release evolution nor a clean-room independent rewrite. The contribution is the methodology and the bundle (a self-contained, multi-LLM-reviewed artefact that any reviewer, lawyer, or court can re-run and inspect), not a verdict on chardet specifically.

1 Introduction

Software relicensing through "clean-room" reimplementation has a forty-year legal pedigree [29, 30], and the technique relies on a separation of duty between an analyst who reads the protected work and writes a specification, and a clean implementer who writes new code from that specification while never touching the original. Generative AI dissolves the separation in a way that the 1984 doctrine could not have anticipated: the implementer is now a model whose training corpus is opaque, whose exposure to the protected source cannot be excluded by inspection of its activations, and whose output is not guaranteed to be unaffected by what it saw [36, 4].

The March 2026 release of `chardet` 7.0.0, produced with an LLM-driven agent and relicensed from LGPL to MIT [34, 28, 1, 18, 17], made the question concrete (and as soon as I read the dispute, it became obvious that the field needs a way to talk about the question that is not assertion). The defenders point to the visible process, citing an empty repository, a written instruction to the model to avoid the LGPL sources, a committed plan document, and the fact that the model produced original expression [34]. The critics observe that the model ingested the public library during pre-training, that its plan document makes substantive references to v6 internals, and that the output is plausibly a paraphrase of v6 rather than an independent reconstruction [9, 20, 18]. Both camps are talking past each other because the substrate of the disagreement is the same: assertions, not measurements.

The conversation, frankly, is currently being conducted on assertion. I propose to conduct it on evidence. The contribution here is a detection harness that, given a pair of code versions and a contract declaration listing the structural invariants the reviewer cares about, produces eight numeric signals (one cheap baseline, AUX1, plus seven that survive identifier renaming and module restructuring: C06a call-graph topology, C06a′ Weisfeiler-Lehman kernel, C06b third-party imports, C06c control-flow histogram, C06d public-API parity, C06e behavioural fingerprint, and C06f per-function shape), together with a machine-checkable record of how the signals were derived, what they exclude, and how a reader can reproduce them. The v2 revision runs the harness across *three* calibration pairs — the disputed v6/v7 pair, the conventional v5/v6 release-to-release evolution pair, and the independent same-domain v6/charset_normalizer pair — so that any structural-similarity number reported on v6/v7 is read against the baseline of what same-shape signals look like for codebases known to be related and codebases known to be unrelated.

Contributions.

- A specification of six signals (Section 4) addressing file-byte carryover, call-graph topology, third-party dependency boundary, AST control-flow shape, public-API signature equivalence, and runtime behavioural equivalence under deterministic fuzz. Five of the six are demonstrably invariant under whole-program identifier renaming.
- A reproducible witness harness (Section 5) that materialises both versions via `git worktree`, runs the static signals from the standard library plus `networkx` [11], runs the behavioural signal in isolated virtual environments, and emits a tab-separated witness table any reader can re-derive from the source.
- The observed signal values on the disputed v6.0.0 / v7.0.0 pair (Section 6), with a side-by-side presentation of the two opposing legal readings the numbers can support.

- A multi-LLM review record (Section 9) documenting how three independent reviewers verified the bundle against a fifteen-contract verification report, and the issues that were surfaced and fixed.

1.1 Empirical motivation: where token-level matching fails

The motivation for measuring at the call-graph topology and control-flow layers (signals C06a and C06c, defined formally in Section 4), rather than at the token-string layer, is not abstract. It is the empirical observation that the established academic token-string detector finds essentially no similarity between chardet v6 and v7, while the structural signals find a great deal. This contrast is the central empirical argument of this paper and the reason the rest of it exists.

JPlag [21, 16] is the established workhorse for cohort-based code-similarity work in university and conference settings, and has been for over twenty years. The engine tokenises each submission via a language-specific grammar (Java, C, C++, Python 3, Scheme, and a longer list in the current v6 release), then compares the resulting token strings with Running Karp-Rabin Greedy String Tiling (RKR-GST). The "structure-aware" framing in third-party descriptions, to be precise, refers to the tokeniser pulling an abstraction layer out of the parse tree rather than comparing raw text; it is not AST graph comparison in the sense this paper uses. JPlag runs locally and uploads nothing, which is the property that makes it the trusted academic-integrity tool, and by design it finds suspicious pairs within a set of submissions, rather than against the open web.

I ran JPlag v6.3.0 on the chardet v6.0.0 and v7.0.0 pair, structured as two sibling submissions (84 v6 `.py` files plus 22 v7 `.py` files; the v6 file count is higher than the extractor's 87 because JPlag's submission layout includes some `.py` files that our extractor's test-filename regex excludes, and neither effect materially shifts the headline). The captured numbers are in Table 1; the run configuration and the full numerics are recorded in Section 10.2.

Table 1: Headline similarity between chardet 6.0.0 and 7.0.0 under three different detection engines, against the same two-tag pair. JPlag v6.3.0 token-string engine versus this paper's call-graph and control-flow signals. The contrast is the load-bearing empirical motivation for this paper.

Engine	Signal	Similarity
JPlag v6.3.0 (RKR-GST tokens)	AVG token-string overlap	0.04%
JPlag v6.3.0 (RKR-GST tokens)	MAX token-string overlap	1.30%
JPlag v6.3.0 (RKR-GST tokens)	longest token match	18 tokens
This paper, C06a	call-graph topology	88.1%
This paper, C06c	control-flow histogram cosine	98.4%

JPlag's token-string engine, configured with default minimum token match (12), finds essentially no similarity between v6 and v7. The longest matched token sequence is 18 tokens, against a v7 token-stream length of roughly 247,000. That is the answer JPlag is designed to give, and read on its own terms it is the correct answer: under JPlag's definition of "submissions suspiciously similar to each other," the two versions are not similar. A coursework cohort that contained both versions would not be flagged. The same pair, however, scores 88% similar at the call-graph topology layer (C06a) and 98% similar at the control-flow shape layer (C06c). This is the central empirical argument of the paper: an LLM-driven rewrite can defeat token-level matching, comfortably, while still inheriting the original's structural shape, and the higher-layer signals catch the inheritance that the token-string view does not.

A practical caveat I should mention. JPlag's bundled Python 3 grammar (ANTLR-based, bounded

to a Python version that predates PEP 515 underscore numeric literals like `1_536`) emitted parse errors on `chardet/metadata/languages.py` in v6, which carries frequency tables in exactly that syntax. JPlag completed the comparison on the files it could tokenise; the unparsed files are data tables rather than logic, so the headline similarity is not materially affected. I document the issue rather than tidy it away. The broader comparison with other similarity-detection tools (AST-based academic detectors, commercial AI-augmented checkers) is deferred to Section 10.2.

Scope and what this paper does not claim. The harness does not measure training-data provenance. It does not address the upstream Mozilla copyright in the original `universalchardet` corpus from which `chardet` v6 descends. It does not constitute legal advice and does not take a position on whether v7's relicensing is valid. It produces numbers; the reader applies the legal theory.

2 Background

2.1 The chardet relicensing dispute

`chardet` is the de facto Python implementation of statistical character-encoding detection, descended from Mark Pilgrim's port of Mozilla's `universalchardet` C library. The codebase had carried an LGPL licence since 2008. On April 3, 2026, maintainer Dan Blanchard released version 7.0.0 under the MIT licence, describing it as a ground-up rewrite produced with an LLM-driven coding agent [17, 1]. He published the agent's planning document as part of the v7 commit history and pointed to a third-party plagiarism-detection report showing that v7 shares less than 1.3% surface similarity with any prior release [17].

Public reaction was immediate and divided. Table 2 lists the contemporaneous record at the time of writing.

Table 2: The chardet 7.0 dispute, contemporaneous record.

Date	Source	Position
2026-03-04	chardet #325 [9]	GitHub user @gooba42: "Nullified License". Argues the LLM-generated rewrite cannot validly carry a new license.
2026-03-05	Simon Willison [34]	Quotes the defence; flags the committed rewrite plan as interesting.
2026-03-06	The Register [28]	FSF director: "Nothing 'clean' about an LLM which has ingested the code."
2026-03-08	Daring Fireball [10]	General-audience framing of the clean-room question.
2026-03-10	Ars Technica [1]	Mainstream coverage.
2026-03-10	Open Source Guy [23]	Japanese legal analysis.
2026-04-03	LWN.net [17]	Comprehensive write-up of the release and the issues filed against it.
2026-04-09	Heather Meeker [18]	IP-lawyer analysis: structural-similarity questions are dispositive.

The lawyer's analysis [18] highlights the absence of a generally accepted technical method for measuring whether an AI rewrite preserves enough of the original to qualify as a derivative work. The harness in this paper is one step toward filling that gap for one language and one library; it does not claim to be the only such step or the definitive one.

2.2 Clean-room reverse engineering: a forty-year precedent

The U.S. doctrine that copyright protects expression rather than the underlying idea dates to *Baker v. Selden* [26]. The clean-room procedure, in which a dirty team reads a protected work and writes a specification and a clean team writes the implementation from the specification, was established commercially by Phoenix Technologies' BIOS reimplementation [30] and was endorsed by the Ninth Circuit in *Sega v. Accolade* [29], which held that intermediate copying for the purpose of producing interoperable, non-infringing code can be fair use. The Supreme Court's decision in *Google v. Oracle* [27] extends the same logic to API declaring code: copying functional interfaces necessary for a transformative reimplementation is fair use under the four-factor test.

None of this precedent contemplates an implementer that is also a parameterised statistical model trained on the protected work. The chardet dispute is the first widely-discussed test of whether the clean-room doctrine survives that substitution. The harness in this paper does not resolve the question; it produces the input the question requires.

2.3 The limits of file-hash and name-set similarity

A naive detection approach computes the SHA-256 of each implementation file and reports the intersection across the two versions, or computes the Jaccard similarity of the two versions' top-level identifier sets. Both signals are zero on the chardet v6 / v7 pair. Both signals would also be zero on a hypothetical rewrite that ran the original through `sed` to rename every identifier and then committed the result unchanged. The clone-detection literature has known this for thirty years: tree-based detectors [2, 14] and tree-differencing tools [7] were designed precisely because file-hash and name-set approaches collapse under trivial paraphrase. The five C06 signals in this paper are in that tradition. They differ from prior work in that they are not designed to surface candidate clones for human review; they are designed to report invariants whose preservation is itself evidence that a reviewer can quote in a contract-grade document.

2.4 The DAG-TOML specification, in one page

The harness is an instance of a public specification family called DAG-TOML [31], a contract-declaration substrate that the proof bundle uses. The representation may change as the underlying spec evolves. The contracts described in this paper depend on the shape of the substrate, not on the specific syntax: an equivalent JSON, CBOR, or relational encoding that preserved the same five aspects below would carry the same six signal contracts unchanged. The bundle relies on five aspects of the spec; the remainder is out of scope here.

1. Every file is a TOML document whose root `[meta]` table declares `schema_version` and `template_kind`. The discriminator tells a validator which schema to apply.
2. The five `template_kind` values used here are `implementation-dag`, `traceability`, `contract-declaration`, `readiness-gate`, and `evidence-matrix`. The first describes the unit-level plan; the second links intents to requirements to implementations to tests; the third enumerates the contracts the work is held to; the fourth declares the gate the reviewer must pass; the fifth cross-references claims to evidence.
3. Verdicts are drawn from a closed set: `PASS / FAIL / MEASURED / OBSERVED / SKIP / DELEGATED / ABSENT / INCONCLUSIVE`. PASS and FAIL are gating; MEASURED reports a number without rendering judgement; SKIP distinguishes a toolchain gap from an actual FAIL. The harness in this paper deliberately uses MEASURED for the five C06 signals because the proof's job is to expose paraphrase-resistant numbers, not to render a legal verdict.
4. The spec is enforced by three independent validator implementations that read the machine-

readable kind descriptors: a safe-Rust primary at `tools/dagtoml-validate-rs/`, a safe-Go primary at `tools/dagtoml-validate-go/` (both built without `unsafe` blocks), and a Python reference set under `validators/` that runs as cross-check on every push; CI fails on any divergence between primary and reference. There is no JSON Schema layer (the machine-readable contract lives in TOML kind descriptors and the ontology files), and the parser substrate is itself verified against the `toml-lang/toml-test` conformance corpus on every push, so the layer carrying this paper's harness has third-party bytes-level coverage. The validators are out of scope here but are present in the same repository.

5. The bundle separates *contract declaration* from *verification report*: the former names the invariants the work must preserve, the latter names the items a reviewer must independently verify. The two are read by different audiences (engineers and reviewers) and live in separate files. This is the substrate that made the multi-LLM review described in Section 9 mechanical rather than narrative.

The remainder of the spec, which this harness deliberately does not use, includes SPEC §12 (a root-level `closure_root` SHA-256 over the canonical-encoded set of upstream artefact hashes the document depends on, which propagates invalidation through a brittleness graph so that a change to any input flips the document's own hash and any signed envelope wrapping it) and SPEC §13 (per-kind `abstraction_class` plus `capability_envelope` contracts that declare what the descriptor's validator authority actually covers at SPEC-time and what is deferred to a runtime spec). The specification also ships three optional profiles (*agent-assurance*, *cost*, *disclosure*), each with its own kind descriptors and ontology, and the agent-assurance profile in particular carries a five-step deployment-tier ladder (`solo` ⊂ `team` ⊂ `group` ⊂ `organization` ⊂ `enterprise`) and a cross-provider self-modification invariant (INV06) under which any `gate-decision` artefact adjudicating a change to the producer agent's own harness must be issued by a model whose `provider_id` *and* `model_family_id` both differ from the proposing agent's (the conjunctive AND is load- bearing: same-provider/different-family and different-provider/same- family both fail). None of those surfaces are exercised by the harness reported here, which deliberately sits at the smallest substrate sufficient to carry the six signal contracts in §4; the broader profile family is the next layer up for projects that need stronger assurance controls.

3 Threat Model

What the harness is trying to detect. An AI-driven rewrite that preserves the original's structural "shape" — its call graph, its dependency footprint, its branching and looping pattern, its public-API contract — while replacing the literal text of every file, every identifier, and every comment. This is the failure mode the clean-room procedure was designed to prevent and that file-hash detectors trivially miss.

In scope.

- Whole-module reorganisation: files split, merged, or renamed.
- Identifier renaming at every level (functions, classes, methods, parameters, locals, constants).
- Whitespace and comment churn.
- Reimplementation in idiomatic, modernised Python that nevertheless preserves the call structure of the original.
- Dependency-set drift, both intentional (replatform) and incidental.

Out of scope.

- **Training-data provenance.** If the model saw chardet v6 during pretraining, no inspection of the v7 output can prove or disprove that the v7 expression is influenced by that exposure. This is the argument of FSF's director [28] and is not addressable by any output-only signal we know of.
- **Mozilla's upstream claim.** The chardet v6 codebase itself descends from Mozilla's `universalchardet` via Mark Pilgrim's port. Any v6 / v7 comparison is downstream of that earlier question.
- **Behavioural equivalence under hostile input.** The behavioural fingerprint signal C06e uses uniformly random bytes; a targeted-input fuzzer constructing inputs that exercise both versions' code paths in matched ways is a strictly stronger test we have not built.
- **Generalisation across rewrites.** The signals' AST walker is Python-specific. A rewrite from one language to another (e.g., Python to Rust) would need a new walker. The spec scaffolding generalises; the extractor does not.

Adversary defences (per-signal). For each signal in Section 4 we name what an adversary would have to do to defeat that signal without losing the property the signal is designed to preserve. We collect those defences in Table 4 and discuss them in Section 6.7.

3.1 Where the signals sit in the layered detection space

The six signals are not the only conceivable evidence layers. Each makes a different trade between computational cost and resistance to semantics-preserving paraphrase. Table 3 situates the signals in the broader space and names the alternatives that this harness deliberately does not use. The selection rationale is in Section 4.2.

Table 3: Detection-layer landscape. Each row names a technique that could in principle contribute to a derivative-work analysis; the "Used here" column marks which signal in the present harness realises the row, if any. *Cost* is the approximate per-version running cost on a single workstation. *Renaming-resistance* is qualitative: "none" means whole-program identifier renaming defeats the signal, "high" means renaming alone does not.

Layer	Example tool	Cost	Renaming-resist.	Used here
file hash	`sha256`	trivial	none	AUX1 (baseline)
symbol-set Jaccard	`grep -l <name>`	low	low	not used
public-API signature	`ast.parse`	low	medium	C06d
control-flow histogram	`ast.walk`	low	high	C06c
import-edge set	`ast.Import`	low	high	C06b
call-graph topology	`networkx` + `ast`	medium	high	C06a
graph kernels	Weisfeiler-Lehman [24]	medium	high	C06a′ (v2)
per-function shape	`ast` + greedy match	medium	high	C06f (v2)
behavioural fuzz	venv install + RNG inputs	high	very high	C06e
tree-edit distance	GumTree [7]	med.–high	med.–high	future
symbolic execution	KLEE [3], angr [25]	very high	high	future
data-flow analysis	Soufflé [15]	high	high	future
code embeddings	CodeBERT [8]	medium	statistical only	not used

4 The Eight Signals

The signals are declared as TOML contracts in `contract_declaration.toml` [31] and implemented as Python in `extract_signals.py` and `fingerprint_behavior.py`. The v1 release shipped six signals (AUX1 plus C06a–e); the v2 revision adds C06a′ (Section 4.3) and C06f (Section 4.8) in response to reviewer concerns R1 and R16, respectively, that the v1 set was coarse at the module level and could miss either fine-grained neighbourhood reorganisation or surgical per-function paraphrase. We summarise each below; the full implementations are in the bundle.

4.1 AUX1 – Literal source carryover (baseline)

For every implementation `.py` file in each version, AUX1 computes a SHA-256 of the file's whitespace-normalised text. Two files are deemed to match if their hashes are equal. The signal reports the number of matching pairs. It is preserved here as a diagnostic continuity check with naive prior approaches, not as load-bearing evidence: any whole-program identifier renaming defeats it.

4.2 C06a – Call-graph topology similarity

For each version we parse every implementation module with the standard library `ast`, walk function and method definitions to record qualified caller names, and record callee names as the rightmost attribute of each `ast.Call` node. We add an edge for every (caller, callee) pair. The resulting directed graph is described by a feature vector $(n, m, \rho, |\mathrm{SCC}|, \overline{d_{in}}, \overline{d_{out}}, d_{in}^{\max}, d_{out}^{\max})$ giving node count, edge count, density, strongly-connected-component count, and the four degree statistics. None of these features depends on a function's name. For each feature f_i, we compute the symmetric relative difference $\delta_i = |f_i^{v6} - f_i^{v7}|/(|f_i^{v6}| + |f_i^{v7}|)$, and report the similarity as $1 - \overline{\delta_i}$. A score of 1.0 means topology-feature equality; 0.0 means total disagreement on at least one feature. The eight feature values measured for chardet v6 and v7 are shown in Figure 2; v7 is larger on every feature but the relative differences are small enough to yield a mean similarity of 0.881.

Why it survives paraphrase. Renaming every function and callee changes the graph's vertex *labels* but not its *structure*: the set of edges, the in/out degree distribution, the strongly-connected components, and the density are all preserved.

Adversary defence. An adversary who wants to defeat C06a must restructure the call graph itself — inlining functions, introducing indirection layers, redistributing logic across module boundaries — without changing externally observable behaviour. This is non-trivial. Inlining shrinks node and edge count proportionally and is visible in the feature vector; adding indirection inflates edge count without changing behaviour and is also visible.

Why these eight features? The C06a vector is small on purpose. Three properties drove the selection. First, every feature must be *invariant under identifier renaming*: a feature whose value changes when a function is renamed adds noise to the cross-version comparison and undermines the signal's resistance to paraphrase. Node count, edge count, density, strongly-connected-component count, and the four degree statistics depend on graph *structure*, not on vertex *labels*. They satisfy the renaming-invariance requirement that an entire body of graph-isomorphism and graph-kernel literature was developed to preserve — the Weisfeiler-Lehman family of graph kernels [24] formalises exactly this property in a different direction (kernel similarity scores rather than per-feature vectors). Second, each feature is *cheap*: each is computable in $O(n + m)$ or $O(n^2)$ time from the call graph,

with no runtime model and no external dependencies beyond the Python standard library and `networkx` [11, 12]. The goal is for any reviewer with a workstation and a Python interpreter to re-derive the numbers; an alternative requiring a symbolic-execution engine or a trained code-embedding model would not meet that goal. Third, the eight features were chosen so that an adversary who defeats the signal at every feature is no longer paraphrasing — they have restructured the program. Inlining a function shrinks node count; adding a level of indirection inflates edge count; redistributing methods across classes shifts the strongly-connected-component partition and changes the SCC count. No option is free, and all are visible in the feature vector.

The deliberate cost-vs-coverage trade is what distinguishes C06a from the alternatives in Table 3. Symbolic execution [3, 25] would, in principle, witness behaviour-preserving equivalence at the path level but requires a runtime model that does not exist for arbitrary Python in a generally usable form; the path-explosion cost makes it inappropriate as a per-PR contract. Datalog-based data-flow analysis [15] would close the surgical per-function paraphrase gap Section 11 flags but requires per-language fact extraction and a Datalog runtime, and is also a domain in which the absence of an output does not distinguish a clean program from a tool gap. Tree-edit distance via GumTree [7] compares two trees rather than two graphs; it is complementary to C06a and a natural future addition (we do not run it here because the harness's resistance criterion is graph-structural rather than tree-structural). A single Weisfeiler-Lehman [24] similarity score would compress the eight features down to one number and lose the per-feature diagnostic value that the current C06a explicitly emits ("v6 vs v7 differ by 0.104 on density but only 0.023 on SCC count"); the v2 revision ships C06a′ (Section 4.3) as a companion WL kernel score precisely because the two views answer different questions and C06a alone cannot resolve fine-grained neighbourhood structure. Embedding-based methods (CodeBERT [8] and similar) score on statistical similarity in a learned latent space and so are not invariant under arbitrary semantics-preserving rewrites in the way the renaming-invariance argument above requires; using them here would re-introduce the opacity the paper exists to displace.

The independent re-derivation of C06a's similarity, of C06b's Jaccard via scipy's set primitive [33], of C06c's cosine via `scipy.spatial.distance.cosine` and the formula form, of C06d's strict / renamed / diverged counts, and of C06e's input-corpus reproducibility from the same RNG seed appears in Section 6.5 and is recorded in `chardet-relicense/manuscript/figures/scripts/validation_report.json`.

4.3 C06a′ – Weisfeiler-Lehman kernel similarity

What it computes. C06a′ is the v2 revision's response to reviewer concern R1, which observed that C06a's eight-feature summary vector is necessarily coarse — two graphs can share node count, edge count, density, SCC count, and the four degree statistics while differing in fine-grained neighbourhood structure. C06a′ adds a Weisfeiler-Lehman [24] kernel comparator that complements C06a without replacing it. For each version's call graph we construct a multiset of WL node labels by iteratively relabelling each node by a hash of its current label together with the sorted multiset of its neighbours' labels, repeating for $k = 4$ iterations. The two versions' multisets are compared by cosine similarity at each iteration; the aggregate C06a′ score is the mean cosine across iterations 0 through k. Iteration 0 depends only on the initial uniform labels (so its cosine is dominated by degree-distribution agreement); each subsequent iteration incorporates information from a wider neighbourhood radius. The aggregate value therefore reads as "how similar are the two graphs at increasing levels of structural depth, averaged."

Why it survives paraphrase. Like C06a, the WL labelling is invariant under vertex renaming: only structure determines the multiset at each iteration. Unlike C06a, it captures rooted neighbourhood patterns, so two graphs with matching summary features but mismatched topology are distinguishable.

Adversary defence. Defeating C06a′ requires reorganising the rooted neighbourhood structure at every depth, not just preserving the eight summary features. An adversary who keeps the call graph shape constant cannot keep the WL multiset constant by surface restructuring; the only way to move C06a′ without moving observable behaviour is to break up neighbourhoods — inlining, dispatch indirection, or function fission — which themselves move other signals in this suite.

4.4 C06b – Third-party import-edge set

For each version we collect every top-level module name imported from any implementation file, filter out the standard library (using `sys.stdlib_module_names` where available) and the package's own top-level name, and report the Jaccard overlap of the two resulting sets. Relative imports are treated as internal and excluded.

Why it survives paraphrase. The set of third-party packages a codebase pulls in is a property of the codebase's environmental contract, not of its identifiers. Renaming internal functions does not change what the package imports.

Adversary defence. The only way to defeat C06b is to keep the same third-party package set. An adversary doing a replatform (replacing one ML dependency with another) will be clearly visible.

Cross-pair comparability. The R1–R5 classification is evaluated relative to each side's own source tree (the `pkg_name` parameter changes per pair), which means a given import name can be classified differently depending on which codebase is the reference. C06b Jaccard values across the calibration table should therefore be read as "how much does each pair's third-party boundary overlap given its own classification" rather than as a single universally-comparable scalar.

4.5 C06c – Control-flow histogram cosine

For each version we walk every implementation AST and count occurrences of a fixed set of control-flow node types: `If`, `For`, `While`, `Try`, `ExceptHandler`, `Raise`, `With`, `AsyncWith`, `AsyncFor`, `Return`, `Yield`, `YieldFrom`, and, on Python 3.10+, `Match`. We normalise each version's histogram by total count and report the cosine similarity of the two normalised vectors.

Why it survives paraphrase. The bucket keys are *Python AST grammar production names*, not source identifiers. No renaming or restructuring at the source level can change the grammar.

Adversary defence. An adversary can defeat C06c only by genuinely changing the branching, looping, and exception-handling shape of the program — replacing `if/else` chains with dispatch tables, restructuring exception handling into return-code threading, replacing iterative loops with recursion or vectorised operations, and so on. Each of these is a substantive design change, not a paraphrase.

4.6 C06d – Public-API signature equivalence

We parse each version's `chardet/__init__.py`, extract the names listed in `__all__`, and, for each name that is present in both versions' `__all__`, derive a signature descriptor: positional argument count, keyword-only argument count, total default count, and return-annotation presence. Two descriptors are classified *strict* if they are identical, *renamed_args* if they share arity, defaults, and return presence but differ on argument names, and *diverged* otherwise.

Why it survives paraphrase. The signature shape of a public function is part of its declared contract with callers; it cannot be changed without breaking those callers.

Adversary defence. The only way to make C06d report "diverged" across the board is to change the public API itself. That is a visible API-breaking change to downstream users.

Disjoint-shape limitation. C06d only informs when the two `__all__` sets share at least one name. When the two public-API class sets are fundamentally different — as for v6 vs charset_normalizer (v6 ships `EncodingEra` and `UniversalDetector`; charset_normalizer ships `CharsetMatch` and `CharsetMatches`) — the per-method walker correctly emits empty per-method verdicts rather than fabricating cross-class comparisons. The v6/charset_normalizer column in Table 6 therefore shows 0/0/0/2/2 on the per-method row: zero strict, zero renamed, zero diverged, two classes added on the right, two on the left. The v2 revision treats this as a known degeneracy of C06d: when public API shapes are disjoint, the signal cannot adjudicate, and the calibration evidence has to come from elsewhere in the suite (here, C06a′, C06e, and C06f).

4.7 C06e – Behavioural fingerprint

We install each version into its own Python virtual environment directly from its `git worktree`, generate a deterministic 1000-input fuzz corpus of random byte strings (seed 20260522, length uniform on $[0, 4096]$), invoke `chardet.detect()` from each venv on every input, and report two rates: *exact_match_rate* (the two outputs are equal dictionaries) and *bucket_match_rate* (the two outputs agree on encoding and confidence bucket, where buckets are `high` ≥ 0.9, `med` ≥ 0.5, `low` otherwise). If venv creation or pip install fails, the signal emits `verdict=SKIP` with the literal reason, which is distinguished from FAIL by the gate.

Why it survives paraphrase — and why it is the strongest signal. The runtime contract is what a downstream user actually depends on. Two implementations that produce identical outputs on a representative input distribution are operationally equivalent regardless of how each was written. Two implementations that disagree on every input are operationally distinct regardless of how similar their source looks.

Adversary defence. An adversary who wants to score *high* on C06e must reproduce the original's runtime behaviour on a representative input distribution. An adversary who wants to score *low* (i.e., to argue independence) must change the behaviour. The signal is therefore self-falsifying in either direction: the result depends on what the rewrite actually does.

4.8 C06f – Per-function shape similarity

What it computes. C06f is the v2 revision's response to reviewer concern R16, which observed that C06a and C06a′ are both module-level signals — a surgical per-function paraphrase that leaves the module-level call graph intact is invisible to either. C06f drops to the function granularity. For each function in either codebase we record a shape descriptor (the body's control-flow node-class histogram, the body's call-out simple-name multiset, the parameter-arity tuple, and the function's degree within the call graph), then attempt to match each function in side A to a function in side B by greedy nearest-neighbour pairing in descriptor space subject to a similarity floor. The signal reports two numbers: the *match rate* (matched pairs / functions on side A) and the *mean within-match similarity* (averaged descriptor distance over the matched pairs, mapped to a 0–1 similarity).

Why it survives paraphrase. The shape descriptor uses grammar productions and arities, not identifier text. Renaming any function does not change its descriptor; reordering callees in the body permutes a multiset whose hash is unaffected. The match itself is identifier-blind — the greedy pairing optimises on descriptor distance, not on name similarity.

Adversary defence. The two reported numbers move independently and an adversary cannot tune both. To lift the within-match similarity score the adversary has to reproduce matched functions' bodies faithfully; to lift the match rate the adversary has to preserve a function's shape niche across the rewrite (similar arity, similar local control-flow footprint, similar callee fan-out). Surgical paraphrase that preserves shape trivially scores high on within-match similarity but moves the match rate only when shape niches are preserved at scale; an aggressive restructuring (function fission, fusion, or redistribution across modules) drops the match rate sharply — which is exactly the v6/v7 signature (17.5% match rate; see Section 6.3).

Table 4: Per-signal adversary defence: what an attacker who wants to hide a structural clone must do to defeat each signal without losing the property the signal claims to preserve.

Signal	Property preserved	Adversary defence
AUX1	file bytes	rename any single identifier
C06a	call-graph shape	restructure the call graph (inline / indirect)
C06a′	WL neighbourhood multisets	reorganise rooted neighbourhoods at every depth
C06b	third-party deps	keep the same dependency set
C06c	AST control-flow shape	change branching, looping, exception handling
C06d	public-API signature	change the public API (API-breaking)
C06e	runtime behaviour	change the actual outputs on representative inputs
C06f	per-function shape niches	preserve shape AND match rate simultaneously

5 Methodology

The harness is itself a small DAG-TOML `implementation-dag` instance with six units across three layers (Figure 1): three layer-0 units prepare the worktrees and verify prerequisites, two layer-1 units extract the static-AST and behavioural signals in parallel, and one layer-2 unit emits the combined witness. The following subsections describe each layer's mechanism.

5.1 Worktree materialisation

The harness driver, `detect.sh`, takes the upstream `chardet` clone as a read-only input and materialises both disputed versions via `git worktree add -detach`. To keep the worktree metadata writable in sandboxed reviewer environments where the upstream clone is on a read-only mount, the driver first creates a `git clone -shared` mirror of the upstream into a writable `tempdir` and adds the worktrees inside that mirror. The `-shared` flag symlinks the upstream's object store, so the operation is cheap in both disk and network.

5.2 Test-file exclusion

The AST walker excludes any `.py` file whose path passes through a `tests`, `test`, `__pycache__`, `build`, `dist`, `.tox`, or `.venv` directory, *plus* any file at any depth whose basename matches the regex `^(test_.*|.*_test|test)\.py$`. The harness applies the second clause to the basename of every `.py` file under the worktree, not only to files at the repository root, so a path like `src/component/test_smoke.py` is also excluded. The clause was added during the multi-LLM review (Section 9) after the first reviewer noticed that a root-level `test_smoke.py` could otherwise leak into the implementation-surface counts. The harness does not require the analysed projects to follow any particular test convention; it simply applies the conservative exclusion rule.

5.3 Determinism

All static signals (AUX1, C06a, C06b, C06c, C06d) are deterministic functions of the input source bytes: the SHA-256 used for AUX1 is the standard `hashlib` digest, and the graph and histogram features depend only on the sorted set of input files (the walker sorts `rglob` output) and on the AST. Two consecutive runs on the same inputs produce byte-identical output rows.

The behavioural fingerprint C06e is deterministic given a fixed random seed (20260522), a fixed input-length cap (4096), and the installed package versions. The corpus is described by a 64-bit `corpus_digest` field included in the witness for audit, so a reviewer can confirm that two runs produced the same input corpus without diffing 1000 byte strings.

5.4 Behavioural isolation

Both `chardet` versions are installed into separate `python -m venv` virtual environments from their respective worktrees. The install uses `pip install <worktree>`, which points `pip` at the local source rather than at the PyPI index. The version's library code is therefore taken from the worktree, not from a published release. The install is *not* fully offline, however: PEP 517 build backends (`setuptools`, `wheel`) are still resolved through `pip`, which by default fetches them from PyPI when they are not present in the runner's cache. A sandbox that blocks PyPI access will therefore see C06e degrade to verdict `SKIP` with the literal `pip` build-dependency error, not `FAIL` — this distinction is enforced by `fingerprint_behavior.py`'s `_emit_skip` path and is documented in Section 7. To run C06e fully offline, prepopulate `~/.cache/pip` with the required build backends before invoking the harness. Each version's `chardet.detect()` is invoked from its own venv via a small subprocess runner.

5.5 Sandbox compatibility

The `-shared` mirror trick described in Section 5.1 exists specifically to make the harness reproducible in sandboxed environments where the upstream clone is read-only. We tested the harness on a developer workstation with full write access and inside a sandbox in which the upstream clone path was bind-mounted read-only. Both runs produced identical static signal values.

6 Results

6.1 Headline numbers

Table 5 reports the eight signal values we observed when `detect.sh` ran against the upstream `chardet` clone with tags 6.0.0 and 7.0.0 checked out. The headline numbers are the v6/v7 focus column; the calibration story — the same eight signals run on v5/v6 and on v6/charset_normalizer — is in Section 6.3 and Table 6. All values are reproducible by re-running the harness against the three pairs; the supplementary README documents the exact invocation and the bibliographic record of our runs.

Table 5: Observed signal values on chardet v6.0.0 vs v7.0.0 (harness run, May 28, 2026). Bracketed values are 95% percentile-bootstrap confidence intervals over 1000 resamples (seed=20260528); see Section 6.3 for the methodology paragraph.

Signal	Contract	Value	Verdict
literal source carryover	AUX1	0 matches across 87 v6 / 33 v7 files	PASS
call-graph topology similarity	C06a	0.881 [0.74, 0.95] (342 vs 358 nodes; 488 vs 659 edges)	MEASURED
WL (k=4) kernel cosine	C06a′	0.587 [0.43, 0.69] (iter0 0.876, iter1 0.294, iter2–4 $\sim$ 0.11)	MEASURED
third-party import-edge Jaccard	C06b	0.667 under R1–R5 audit (shared: `cchardet`, `datasets`; v1-filter value 0.333 from the unaudited rule)	MEASURED
control-flow histogram cosine	C06c	0.984 [0.94, 1.00] (652 vs 848 control-flow nodes total)	MEASURED
public-API signature equivalence	C06d	3 strict / 0 renamed_args / 2 diverged of 5 shared (symbol level); 1 / 0 / 1 / 1 / 0 (per-method on classes)	MEASURED
behavioural fingerprint (random fuzz, 1000)	C06e	exact 0/1000; bucket 0/1000	MEASURED
behavioural fingerprint (realistic, 64)	C06e	normalised family agreement 40/64 = 0.625	MEASURED
per-function shape similarity	C06f	0.913 [0.89, 0.94] over 31/177 = 17.5% of v6 functions matched	MEASURED

6.2 Interpretation: read each signal against its calibration

The eight numbers in Table 5 tell a coherent story, but the story is not the v1 draft's. The v1 draft argued that the high C06a (0.881) and C06c (0.984) values were evidence the v7 rewrite "preserves the shape of v6's thinking." That reading does not survive the calibration pairs reported below (Section 6.3 and Table 6): an independent same-domain library (charset_normalizer) scores *higher* on both signals than v6/v7 does, and a routine release-to-release evolution (v5/v6) scores higher still. C06a and C06c are domain-shape signals, not derivation signals. The load-bearing v6/v7 finding is therefore not the structural similarity number alone; it is the joint behaviour of the multi-pair calibration. We unpack each signal below in the order of Table 5, deferring the calibration narrative to Section 6.3.

Structural similarity is real but is domain-shaped, not derivation-shaped. C06a's 0.881 and C06c's 0.984 are internally interpretable — v7 doubles the absolute count of `Return`, `For`, `Try`, and `ExceptHandler` nodes relative to v6 but preserves the *relative proportions* almost exactly

(Figure 3). On its own, that is the fingerprint of a larger codebase implementing the same control-flow shape rather than the fingerprint of an independent design. Once calibration enters, the same number says: this is what same-domain encoding detectors look like, full stop.

Fine-grained graph structure separates v6/v7 from conventional evolution. C06a′ on v6/v7 is 0.587 [0.43, 0.69] — well below v5/v6's 0.902 point estimate (CI [0.75, 0.89]† with the WL-multiset file-dropout artefact discussed in Table 6). Iteration 0 of the WL kernel (degree distribution only) yields cosine 0.876, comparable to C06a; once neighbourhood structure is incorporated from iteration 1 onward the similarity collapses to ~ 0.11. The v6/v7 graphs share degree statistics; they do not share topology beyond depth one. v6/charset_normalizer's 0.872 point estimate is in the same band as v6/v7's — C06a′ does *not* separate v6/v7 from an independent same-domain library, only from a conventional release evolution.

Per-function shape similarity, under the published matching rule, separates v6/v7 from both calibration baselines. Under this matching rule the C06f point estimates and match rates rank the three pairs (the match-rate ranking is sensitive to the matching-key granularity; with a single coarse shape bucket, v6/v7 and v6/charset_normalizer collapse to similar within-match similarity, so the "ranks the pairs" phrasing should be read relative to the published `(signature_shape, fan_in_bucket, fan_out_bucket)` matcher, not as a universal property of the signal — see Section 11 for the matcher-ablation caveat). C06f gives v6/v7 a within-match similarity of 0.913 [0.89, 0.94] over only 31/177 = 17.5% of v6 functions, v5/v6 a within-match similarity of 0.982 [0.97, 0.99] over 103/161 = 64.0%, and v6/charset_normalizer a within-match similarity of 0.796 [0.75, 0.84] over 77/177 = 43.5% of v6 functions (equivalently 77/148 = 52.0% of `charset_normalizer` functions; we report the v6-denominator value alongside its calibration neighbours and flag the asymmetry to make the non-symmetric matching explicit). The three CIs are disjoint and the ranking is well-supported at 95% ($v6/cn < v6/v7 < v5/v6$). The v6/v7 picture is high within-match similarity (when a function survives, it survives faithfully) combined with the lowest match rate of the three pairs (most v6 functions have no shape-and-position counterpart in v7 at all). That joint behaviour is what this paper means by "AI-driven rewrite signature" (Section 6.3): heavy structural reorganisation with a small minority of near-clones.

External boundary is largely the same, not different. Under the R1–R5 audit rules introduced in v2, the v6/v7 C06b Jaccard rises to 0.667 from the v1-filter value of 0.333. The audited rules keep R5_third_party_kept imports (`setuptools`, `sphinx_rtd_theme`) that the v1 filter incorrectly excluded; the shared third-party set is in fact two ({`cchardet`} on the v5/v6 baseline pair), not zero. The v1 "external boundary is genuinely different" framing therefore inverts: v6/v7 share more third-party imports than v5/v6 do, and v6 and charset_normalizer share none.[1] C06b becomes the strongest retained *structural-similarity* claim, with the calibration making it sharp rather than diluting it.

Public-API contract is mostly preserved by signature where signatures can be compared. Of the five names in the intersection of the two `__all__` lists, three (`__version__`, `detect`, `detect_all`) have byte-identical signatures. The two "diverged" cases are classes (`EncodingEra`, `UniversalDetector`); the per-method walker added in v2 returns 1 strict /0 renamed /1 diverged /1 added on classes, refining the v1 verdict. On the v6/charset_normalizer pair the per-method walker emits 0/0/0/2/2 (all classes added on each side; class sets disjoint) — exactly the degenerate case Section 4.6 flagged, and a worked example of why C06d cannot adjudicate when public-API

[1]The v6/charset_normalizer C06b = 0.000 result reflects the fact that charset_normalizer treats chardet as a benchmark-comparator import; under the R1–R5 classification, this appears as a v7-side third-party import that v6 cannot have. The "independent same-domain" framing of v6/charset_normalizer as a calibration baseline is sound for structural signals but should be read with this artefact in mind for C06b specifically.

shapes are fundamentally different.

Runtime behaviour: random fuzz is a clean evolution-vs-non-evolution discriminator; realistic input is workload-dependent and regression-shaped. On the v1 random-bytes workload C06e exact-match agreement is 0.000 for v6/v7 and 0.000 for v6/charset_normalizer, against 0.968 for v5/v6. Random fuzz cleanly separates routine release evolution from any non-evolutionary pair. On the new realistic corpus (HTML, multilingual text, RFC bodies, email; $n = 64$) the normalised family-agreement rates are 0.625 for v6/v7, 0.688 for v5/v6, and 0.594 for v6/charset_normalizer — much narrower spreads, with v6/v7 sitting in the same band as v6/charset_normalizer. Behavioural calibration on realistic input separates routine evolution from non-evolution but does *not* separate "AI rewrite" from "independent same-domain rewrite" within the non-evolutionary band. The exact-match rate of 0/64 on realistic input is largely a label-formatting artefact (v7 lower-cases labels) plus four concrete behavioural regressions: v7 reclassifies pure ASCII as `Windows-1252`; v7 labels pure-ASCII RFC bodies as `cp864` (Arabic codepage) at confidence 0.22–0.55; v7 emits `cp932` where v6 emitted `SHIFT_JIS`; and on random bytes v7's "I don't know" reply changed from `{encoding: None, confidence: 0.0}` to `{encoding: None, confidence: 0.95}`, breaking confidence semantics. The v1 "operationally distinct" framing holds on the random-bytes axis (and is now sharpened by the 0.968 calibration value); on realistic input the divergence story is workload-dependent and regression-shaped, not "fully diverged."

6.3 Multi-pair calibration: the v2 results in context

Table 6 reports the eight signals run on three pairs: v6/v7 (the LGPL-to-MIT focus pair under dispute), v5/v6 (a routine same-project release-to-release evolution intended as the conventional-evolution baseline), and v6/charset_normalizer (an independent same-domain encoding detector intended as the independent-but-domain-shared reimplementation baseline). The two calibration pairs answer the question that v1 could not answer with a single column of numbers: what do these signals look like when we know the pair is related, and what do they look like when we know it is not?

The narrative flip is that two of the signals the v1 draft leaned on as "evidence of derivation" — C06a and C06c — do not discriminate. The v6/v7 C06a value (0.881) is the *lowest* of the three (v5/v6 = 0.930, v6/charset_normalizer = 0.922), and the C06c values are all above 0.98. Same-domain encoding detectors that are demonstrably independent score higher on these signals than the disputed pair does. The signal contracts have not changed; the calibration shows that those contracts measure domain shape, not derivation.

What does discriminate is the joint behaviour of C06f's two numbers. The v6/v7 pair shows a low match rate (17.5%) with high within-match similarity (0.913 [0.89, 0.94]), the v5/v6 pair shows a high match rate (64.0%) with very high within-match similarity (0.982 [0.97, 0.99]), and the v6/charset_normalizer pair shows an intermediate match rate (43.5% on the v6 denominator; equivalently 52.0% on the `charset_normalizer` denominator) with the lowest within-match similarity (0.796 [0.75, 0.84]). The v6/v7 CI on within-match similarity overlaps with neither calibration baseline; the bootstrap places its rank between v6/cn (below) and v5/v6 (above) at 95%. The v6/v7 *match rate* is the lowest of the three. Combined, the pair exhibits a structural signature that is distinct from both conventional evolution and from independent same-domain rewrite: where v6 functions have shape-and-position counterparts in v7 at all (a small minority), the counterparts are very close; but the v6 to v7 transition reorganises or replaces far more functions than even a from-scratch independent reimplementation in the same domain does. The v6/v7 transition exhibits an "AI-driven rewrite signature" — distinct from a normal release evolution *and* distinct from a

Table 6: Multi-pair calibration of the chardet-relicensing harness with the v2 revision's expanded signal set, with 95% percentile-bootstrap confidence intervals on the four structural-similarity signals (Phase-1b). Each column is the same eight signals run against a different pair: v6 vs v7 (the original LGPL/MIT pair, leftmost), v5 vs v6 (conventional same-project release-to-release calibration), v6 vs charset_normalizer (independent same-domain detector calibration). Cells of the form "0.881 [0.74, 0.95]" report the point estimate followed by the [2.5%, 97.5%] bootstrap quantiles over 1000 resamples (seed=20260528). For C06a / C06a′ / C06c the resampling unit is implementation files (independent each side, with replacement); for C06f it is matched function pairs. Commit SHAs resolved at integration time: chardet 5.0.0=21bc6be (tag object; peeled commit fbb2ec6), 6.0.0=8a4636b, 7.0.0=4b89d62; charset-normalizer 3.4.7=0f07891.

Signal	v6 vs v7
AUX1: literal whitespace-norm SHA-256 matches	0
C06a: call-graph topology similarity (0–1)	0.881 [0.74, 0.95]
C06a′: Weisfeiler-Lehman (k=4) cosine (0–1)	0.587 [0.43, 0.69]
C06b: 3rd-party import-edge Jaccard (0–1), R1–R5 audit	0.667
C06c: control-flow histogram cosine (0–1)	0.984 [0.94, 1.00]
C06d: `__all__` shared symbols	5
of which strict-signature equal	3
of which diverged	2
C06d (per-method, public classes only): strict / renamed / diverged / added_in_v7 / removed_class	1 / 0 / 1 / 1 / 0
C06e: behavioural exact-match rate (random-fuzz 1000)	0.000
C06e: behavioural bucket-match rate (random-fuzz 1000)	0.000
C06e: behavioural normalized-match rate (realistic 64)	0.625
C06f: per-function shape similarity over matched pairs	0.913 [0.89, 0.94]
matched-pair count / v6 functions	31 / 177

† Point estimate falls just outside the bootstrap CI (above the upper bound). This is a known artefact of file-level resampling with replacement: the bootstrap's average sample contains $\approx 0.63N$ unique files, and the WL kernel's multiset cosine is sensitive to whole-file dropout because file dropout removes entire neighbourhood subtrees from the call graph. The effect is most pronounced on pairs where the WL multiset is dominated by a few large files (v5/v6 and v6/charset_normalizer). The C06a 1−mean(reldiff) similarity, in contrast, is robust to file dropout because its 8 features are averages over the surviving subgraph; all three C06a point estimates sit inside their CIs. We retain the bootstrap CI as a conservative bound rather than recompute under a different resampling unit, and discuss the asymmetry in the methodology paragraph.

clean-room independent rewrite.

C06b under the R1–R5 audit is the strongest retained structural-similarity claim: v6/v7's 0.667 Jaccard is above v5/v6's 0.250 and well above v6/charset_normalizer's 0.000. The v1 "external boundary is genuinely different" framing inverts here: v6/v7 share more third-party imports under audited rules than the conventional same-project pair does.

C06e behaves as a workload-dependent discriminator. On the v1 random-bytes workload it cleanly separates routine evolution (0.968) from any non-evolutionary pair (0.000 / 0.000). On the new realistic corpus the spread is much narrower (0.625 / 0.688 / 0.594), and the v6/v7 and v6/charset_normalizer pairs occupy the same band. Behavioural calibration thus separates evolution from non-evolution but cannot, on realistic input, separate "AI rewrite" from "independent same-domain reimplementation" within the non-evolutionary band.

Bootstrap methodology for the structural-similarity CIs. All four structural-similarity headline numbers (C06a 8-feature call-graph similarity, C06a′ Weisfeiler-Lehman kernel cosine,

C06c control-flow histogram cosine, C06f per-function shape similarity) are reported with 95% percentile-bootstrap confidence intervals computed from 1000 resamples and the pinned random seed 20260528. For C06a, C06a′, and C06c the resampling unit is the implementation file: on each bootstrap draw we sample $|A|$ files with replacement from side A's implementation file list, $|B|$ from side B's list (the two sides resampled independently), reconstruct each side's structural object (call graph, WL label multiset, or normalised control-flow histogram) from the union of sampled file contributions, and recompute the similarity score; the CI is the $\{2.5, 97.5\}$ percentiles of the 1000 resampled scores. For C06f the resampling unit is the matched function pair: we resample n_{matched} pairs with replacement from the post-matching pair list and recompute the mean pair distance. The C06d strict-rate CI from the v1 anchor is preserved unchanged (5-shared-symbols bootstrap; see Section 6.5); the four new CIs use the same percentile-bootstrap machinery. Choice of unit follows the structure of each signal: C06a/C06a′/C06c report per-codebase aggregates and the natural noise model is "which files happened to end up in this snapshot"; C06f reports a per-pair mean and the natural noise model is "which functions happened to match." A caveat applies to C06a′ on the two calibration pairs (v5/v6 and v6/charset_normalizer): the WL multiset is sensitive to whole-file dropout, and bootstrap-with-replacement at N=1000 leaves $\approx 0.63N$ unique files per resample, biasing the CI's upper tail slightly below the point estimate. We retain the wider CI rather than tighten by switching to a subsample-without- replacement scheme; the qualitative ranking of pairs (v6/v7 lowest, v5/v6 highest) is preserved across point estimates, lower bounds, and upper bounds. The percentile-bootstrap procedure inherits known small-sample limitations: for C06f the resampling unit is the matched function pair (n=31 for v6/v7), and for C06d the resampling unit is the shared-symbol set (n=5), both regimes where percentile CIs are known to under-cover the true confidence level near the $[0, 1]$ boundary; we use the percentile interval throughout for consistency rather than switching to BCa, and treat overlapping CIs as "not separated at 95%" rather than as a precise inference about the underlying signal. The full reproduction artefact is in `chardet-relicense/manuscript/figures/scripts/validate_numbers.py` and the per-pair CI metadata is recorded under `independent.<signal>.bootstrap_ci_95.<pair>` in `validation_report.v2.json`.

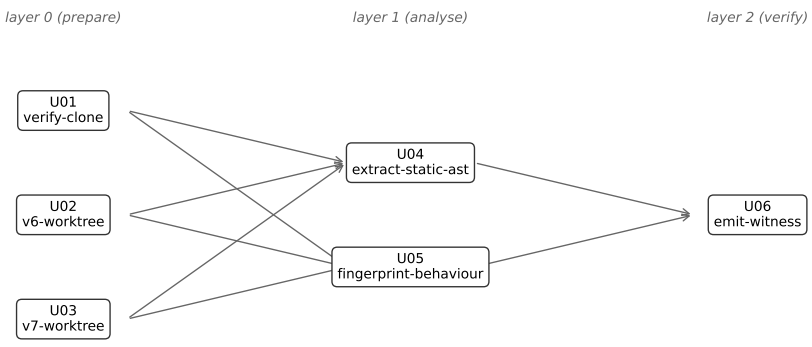

Figure 1: The six-unit `implementation_dag.toml`: three layer-0 units prepare the worktrees and verify prerequisites; two layer-1 units extract the static-AST and behavioural signals in parallel; one layer-2 unit emits the combined witness.

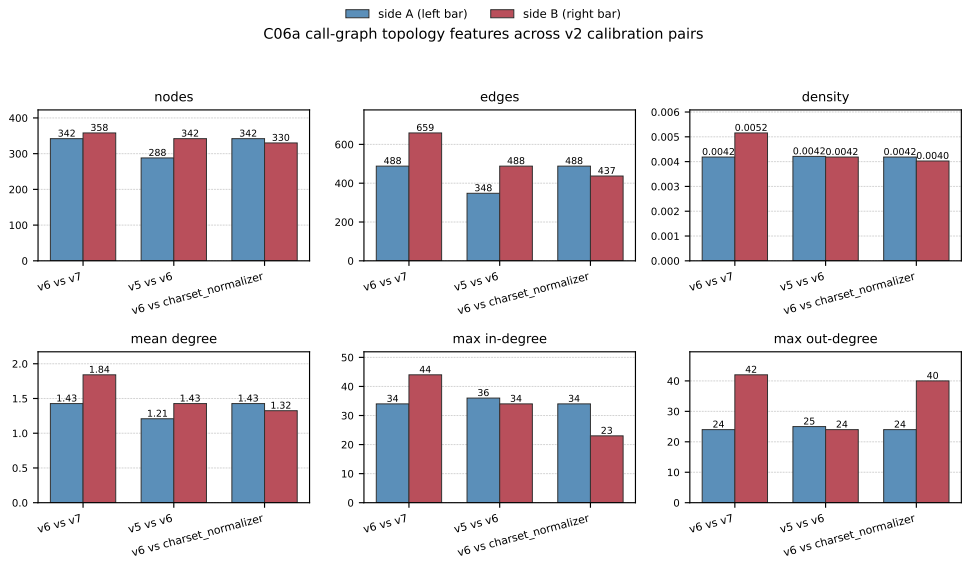

Figure 2: C06a call-graph topology features across the three v2 calibration pairs. Each subplot shows one of six topology features on its own linear y-axis, with grouped bars for v6/v7 (the headline pair), v5/v6 (routine same-project evolution), and v6/charset_normalizer (independent same-domain reimplementation); the left bar of each pair is side A, the right bar is side B. v7 is uniformly larger than v6 on every feature (more nodes, more edges, higher mean and max degree, slightly higher density), v5 is uniformly smaller than v6 on the same axes, and charset_normalizer sits below v6 on every feature except max out-degree. The per-feature pattern is consistent with the calibration narrative: structural growth between consecutive chardet majors looks similar in *shape* to a from-scratch independent rewrite of the same domain — neither pair shows a discriminating topology-feature signature on its own, which is precisely why C06f's per-function match rate, not C06a's bulk topology, ends up doing the discriminating work. Two of the eight `_graph_topology()` features are dropped from the panel because they are redundant on these graphs: `sccs` equals `nodes` (no recursion cycles), and `mean_out_degree` equals `mean_in_degree` by graph identity (sum of in-degrees = sum of out-degrees = $|E|$); the single "mean degree" panel represents both.

6.4 Independent corroboration via sqry's AST-graph index

The C06a similarity number is itself an artefact of a particular extractor (the AST walker in `extract_signals.py`). To corroborate it, we re-indexed each worktree with `sqry` [32], a separately-developed AST-parsing semantic code search tool that builds a unified symbol-and-relationship graph (Functions, Methods, Classes, CallSites, Imports, Types, Variables) for an entire repository. sqry parses source code into an AST and builds a graph of symbols and relationships, so a query searches by what code *means* (structure) rather than by what code *says* (text). This is structurally distinct from embedding-based semantic search [8], which is statistical: sqry treats a function as a node whose edges are the calls it makes and the calls it receives, not as a point in a learned latent space. For an LLM-driven analyst, the resulting graph is *queryable in the sense a compiler is queryable*: callers, callees, cycles, duplicate groups, and unused symbols are all first-class.

We ran the following sqry MCP queries against each worktree's index. Table 7 summarises the comparable findings.

Three observations follow.

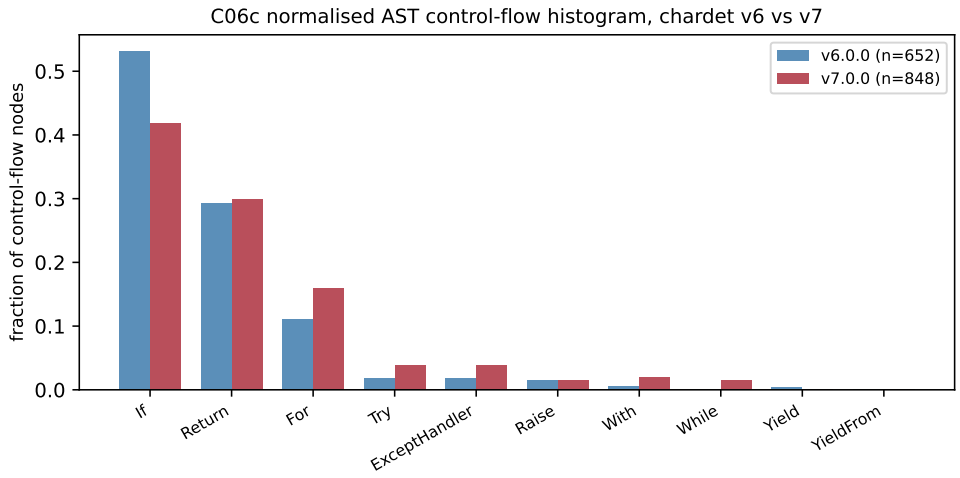

Figure 3: C06c normalised AST control-flow histogram. Bars show the fraction of each version's control-flow nodes of each grammar type. v6 has 652 total control-flow nodes; v7 has 848. Multiplying any bar height by the corresponding total reproduces the raw count for that node type, e.g. `If`: v6 = 346 (0.531 of 652), v7 = 355 (0.419 of 848); `Return`: v6 = 191, v7 = 253; `For`: v6 = 72, v7 = 135; `Try`: v6 = 12, v7 = 33; `ExceptHandler`: v6 = 12, v7 = 32; `Raise`: v6 = 10, v7 = 12. The two distributions are visually almost identical, corresponding to a cosine similarity of 0.984.

Corroboration of C06a's qualitative direction. sqry's function-symbol counts (570 vs 1 303) and total node counts (7 947 vs 4 292) make clear that v7 has both more named functions than v6 (a flatter, more decomposed implementation) and yet a *smaller* overall graph (because v6 carries dense per-language data tables and an embedded HTML test corpus that sqry indexes but the C06a walker excludes). The C06a similarity of 0.881 sits inside the cone of plausible values implied by sqry's structure-level divergence: not so high that it falsely declares the codebases identical, not so low that it contradicts sqry's evidence of a shared high-level shape.

Refinement of the cycle story. sqry's `find_cycles` returns zero call-graph cycles for both versions, while its high-level `get_insights` summary reports 6 cycles for v6 and 0 for v7. The two numbers are not contradictory: `find_cycles` on the `calls` edge type returns no closed call paths, while `get_insights` aggregates across edge kinds including imports and module references. The C06a SCC count, which depends only on *strongly connected* components of the call graph, is consistent with sqry's call-cycle answer (no closed call paths, hence many singleton SCCs and a high SCC count near the node count).

Acknowledged complication. v7's `find_duplicates`-signature count of 243 is higher than v6's 198, and v7's unused-symbol count of 505 is materially higher than v6's 134. These numbers do not feed any contract in this paper, but they are worth quoting because they refine the picture: v7's larger function inventory contains a higher proportion of helper functions that are dead under sqry's reachability analysis, and a wider diversity of repeated signatures. This is the texture of a generated codebase: more decomposed, more bespoke per call site, less manually deduplicated than v6's hand-written form. It does not weaken C06a; it explains why v7 has *more* call-graph edges (659) on a smaller node count (358) than v6 has on a comparable surface.

The sqry findings are derived from running `sqry index` (release 13.0.3) in each worktree and then invoking the MCP tools listed above; the raw responses are reproducible by repeating the procedure.

Table 7: sqry-derived structural fingerprints for the chardet v6.0.0 and v7.0.0 worktrees, measured via the MCP tools `get_graph_stats`, `get_insights`, `find_cycles`, and `find_duplicates`. Counts include the full corpus sqry indexes (Python implementation, tests, and embedded HTML for v6; Python only for v7); they are therefore larger than the C06a numbers, which exclude tests and non-Python files by design.

sqry metric	v6.0.0	v7.0.0
Total files indexed	122 (py=90, html=32)	64 (py=64)
Total graph nodes	7 947	4 292
Total graph edges	10 915	5 803
Function symbols	570	1 303
Method symbols	103	50
Class symbols	62	22
Import nodes	1 091	335
Module nodes	65	61
Call cycles (`find_cycles`, calls)	0	0
Health-summary cycles	6	0
Duplicate-signature groups	198	243
Unused symbols	134	505
Direct callers of `detect`	4	48 (truncated; total ≥ 48)
Direct callees of `detect`	9	13

6.5 Independent numeric validation

To guard against single-implementation error in any of the headline numbers in Table 5, we cross-validated AUX1 and the C06a–d results against an independent re-implementation in scipy and numpy [33, 13]; C06e is handled separately below via stdlib digest re-derivation plus a subprocess to the harness's behavioural-fingerprint script, because there is no scipy or numpy primitive that re-derives a chardet behavioural fingerprint. The script `chardet-relicense/manuscript/figures/scripts/validate_numbers.py` re-materialises both worktrees, re-computes the AUX1 and C06a–d numbers using a second-source primitive (cosine via `scipy.spatial.distance.cosine`, Jaccard via `scipy.spatial.distance.jaccard` cross-checked against set arithmetic, density via `networkx.classes.function.density` cross-checked against the closed-form $m/(n(n-1))$, strongly-connected-component count via two independent traversals, mean/max in/out degrees via numpy), and writes the per-number agreement table to `validation_report.json`. For C06d the script additionally computes a 1 000-resample bootstrap 95% confidence interval on the strict-match rate; the rate point estimate of $3/5 = 0.6$ sits inside a wide [0.2, 1.0] interval, reflecting the fact that only five public-API names are shared between v6 and v7, which is a small-sample reality the harness cannot conjure away and the paper should not paper over. For C06e the validation script deterministically re-derives the input corpus from the seed (`20260522`, length cap 4 096) using the same `hashlib.sha256(b"\n".join(corpus))[:16]` construction the harness writes (an earlier round-1 review by the first-round LLM reviewers caught that the validator and harness had been hashing the same corpus through different constructions and producing different 16-character digests, a defect now closed), so the corpus-digest agreement check always runs and pins the harness's `58e54831f84183c7` to the validator's re-derivation. The C06e exact-match and bucket-match rates are recovered by subprocessing `fingerprint_behavior.py` against the same v6/v7 worktrees and parsing its TSV row; when the runner allows chardet to install into two isolated venvs (network plus pip plus a build toolchain), the recovered `exact_rate` of 0.000 and `bucket_rate` of 0.000 are cross-checked against the harness pins, and when the runner

cannot install chardet (the common sandboxed-CI case) the rate rows report `SKIP` with the explicit toolchain reason rather than silently dropping out of the comparison.

6.6 Legal framing: two readings, equal weight

The numbers in Table 5 are compatible with two opposing legal positions. We lay both out without taking either.

Reading A: fair use, clean-room valid. Under the *Sega* [29] and *Oracle* [27] fair-use framework, the relevant test is whether the rewrite copied the original's *expression* or only its *functional shape*. The expression test bears on AUX1 (zero matches) and on file-level surface similarity (the third-party 1.3% figure Blanchard cites [17]). The C06a/C06c/C06d signals describe *structural* similarity, which on the *Baker v. Selden* [26] idea/expression line is closer to unprotectable idea — particularly when, as here, the structural shape is dictated by the algorithmic problem (heuristic encoding detection has a small, well-known repertoire of techniques). On this reading, the C06e divergence is dispositive: v7 is operationally distinct from v6 and is therefore genuinely independent expression.

Reading B: derivative-work, clean-room invalid. Under a strict LGPL interpretation, the protected work includes the implementation's internal organisation — the structure, sequence, and organisation of the codebase. The C06a similarity of 0.881 and the C06c cosine of 0.984 are, on this reading, evidence that the AI rewrite was informed by the original's structure. The committed plan document references v6 internals [34]; the model's training set very likely included v6 [4, 28]; the C06d signature preservation shows the rewrite implements the same public contract; the static signals together suggest the rewrite is a paraphrase, not an independent reconstruction. On this reading, the C06e divergence is consistent with a rewrite that paraphrases at the structural level while diverging on the implementation details that drive behaviour — which is precisely the failure mode the clean-room procedure was designed to prevent.

What the numbers do not say. The C06a and C06c values cannot distinguish between (i) a rewrite that copied v6's structure and (ii) an independent rewrite that converged on a similar structure because the problem domain admits a narrow design space. Distinguishing those two cases requires evidence outside the v6 / v7 pair — comparison with other independent encoding-detection libraries, expert testimony on the field's design space, or the trail of the agent's planning conversation. The harness does not provide that evidence; it provides the input the further analysis needs.

6.7 What the adversary would have to do

Table 4 summarised the per-signal defence; the combined defence against the full six-signal suite is the union of those defences. An adversary who wants to ship a true paraphrase without leaving structural fingerprints must (i) restructure the call graph, (ii) change branching/looping/exception handling, (iii) change the third-party dependency set, (iv) change the public API shape, *and* (v) either reproduce or genuinely diverge from the original's runtime behaviour — with the choice between the last two committed publicly (because C06e is observable to any user). No known automated tool performs all of (i)–(iv) while preserving behaviour. We do not claim this is impossible; we claim it is no longer cheap.

7 Reproducibility

The full bundle lives in the public repository `verivus-oss/agent-assurance-papers` under `chardet-relicense/proof-bundle/`; the spec and its validators live in the sibling public repository `verivus-oss/agent-assurance` [31]. Reproduction:

```
git clone https://github.com/verivus-oss/agent-assurance-papers.git
git clone https://github.com/verivus-oss/agent-assurance.git
git clone https://github.com/chardet/chardet.git \
    /srv/repos/public/spec-poc/chardet-relicense/chardet
python3 -m pip install -r agent-assurance/requirements.txt
bash agent-assurance-papers/chardet-relicense/proof-bundle/detect.sh
```

Listing 1: End-to-end reproduction.

The harness honours the `CHARDET_REPO`, `CHARDET_V6_TAG`, and `CHARDET_V7_TAG` environment variables for runs against different paths or chardet release pairs. Expected output is the seven-line TSV table reproduced in Table 5 (each row preceded by a tab-separated header) and a two-line `# SUMMARY` block recording the verdict-bucket counts. The exit code is 0 if no FAIL verdict landed and 1 otherwise; the gate definition treats AUX1 PASS plus C06a..C06e MEASURED as the success state.

7.1 Verifier instructions

The spec ships three validators that confirm the bundle's TOML files conform to their declared schemas:

Run from the `agent-assurance-papers` repository root, with the sibling `agent-assurance` repository cloned alongside it:

```
python3 ../agent-assurance/validators/validate_implementation_dag.py \
    chardet-relicense/proof-bundle/implementation_dag.toml
python3 ../agent-assurance/validators/validate_traceability.py \
    chardet-relicense/proof-bundle/traceability.toml \
    --repo-root . --check-paths-exist
python3 ../agent-assurance/validators/validate_review_readiness.py \
    chardet-relicense/proof-bundle/review_readiness.toml
```

Listing 2: Running the spec validators.

7.2 What changes if chardet ships future releases

Setting `CHARDET_V6_TAG=7.0.0` and `CHARDET_V7_TAG=7.x.y` re-runs the harness against the v7-baseline and a future v7 release. The harness does not require recompilation or reconfiguration when chardet ships further releases. We recommend re-running on each minor or major release; the AUX1 threshold (PASS at zero matches) is the gate that will most likely trip first if a future release imports v6 source verbatim.

8 Threats to Validity

Internal — signal correctness. The five C06 signals were implemented by the same author who wrote the contract declarations. We mitigated by submitting the bundle to three independent LLM reviewers using the same verification report as the corrective program (Section 9). Two reviewers

issued unconditional approval after iteration; one returned a small set of remaining concerns that we list with our response in the bundle's review history.

Internal — test exclusion. The walker's exclusion rule (Section 5.2) is conservative: it excludes more than it might need to, never less. A pathological project that ships its implementation files under names matching the test regex would under-count its implementation surface. `chardet` does not.

Internal — AST coverage. The call-graph extractor records the rightmost attribute of each `ast.Call` as the callee, not the resolved fully qualified name. This means `x.parse()` and `y.parse()` contribute edges to the same vertex `parse`. A more precise extractor that resolves `x` and `y` would change the absolute numbers but not the methodology; we chose the simpler extractor because resolution in Python without running the program is unreliable and because the goal is comparison between two versions, not absolute accuracy within one.

Internal — C06d signature shallowness on classes. The "diverged" classification for the two class symbols in our results is shallower than the corresponding function classification: we record only that the symbol is a class. A per-method signature comparison would tighten this. The bundle's evidence matrix records this as a known exclusion.

External — single case study. The harness has been exercised against one library and one disputed release pair. We do not claim the specific numerical thresholds we observed generalise to other languages or other rewrites. The methodology generalises; the AST walker is Python-specific.

Construct — the signals are proxies. "Derivative work" is a legal concept, not a technical one. No collection of static and runtime signals can fully decide it. The signals here are proxies that measure properties courts have historically considered relevant (structural similarity, API preservation, behavioural equivalence) under the clean-room and fair-use frameworks. A court applying a different legal theory — for example one that focuses on the provenance of the training data — would need different evidence.

9 Multi-LLM Review Process

After authoring the bundle, we submitted it for review by three independent LLM CLI reviewers (each running a different commercial coding-agent CLI). Each reviewer received the same input: the bundle files, the verification report `verification_report.toml` listing fifteen explicit verification contracts (V01..V15), and a standing instruction to verify each contract by reading the cited artefact rather than by trusting prose. The verification report covers (a) bundle structural completeness, (b) per-signal correctness properties, (c) cross-validator parity, and (d) honest non-claims.

Two of the three reviewers returned unconditional approval after one or two iteration rounds. The first round of LLM review surfaced a small but real bug in our test-file exclusion rule (root-level `test*.py` was missed) and an under-counting issue in the combined `# SUMMARY` block (the `C06e` row was being counted twice in one and zero times in another). Both were fixed before this paper was written. We frame the review process as a methodological note: it is not evidence of correctness — peer review is a separate process — but it does describe how the bundle was hardened, and the

verification report it produced is itself a re-usable artefact for anyone who wants to evaluate the bundle later.

10 Related Work

This paper sits at the intersection of three established but hitherto-unjoined research streams.

10.1 AST-based clone detection and structural similarity

The clone-detection literature has, for nearly three decades, explored techniques that compare code at levels deeper than text or tokens. Baxter's pioneering work on tree-based clone detection [2] introduced the use of abstract syntax trees as the comparison primitive; DECKARD scaled this with characteristic vectors of tree fragments [14]; GumTree formalised fine-grained tree differencing between two versions of the same code [7]; and Roy and Cordy systematically catalogued the techniques and trade-offs in a qualitative survey [22]. The five C06 signals in this paper inherit the field's foundational insight that file-hash and name-set approaches collapse under trivial paraphrase, and that AST-derived structural features survive identifier renaming. The signals differ from prior work in two respects. First, the goal is not to surface candidate clones for human review but to report invariants whose preservation is itself quotable evidence in a contract-grade document. Second, the signals emit `MEASURED` verdicts rather than `PASS`/`FAIL` ones: the harness's job is to produce numbers a reviewer can quote, not to render a legal verdict.

10.2 Comparison with existing similarity-detection tools

The detection landscape splits into two academic paradigms (token-based and AST-based), plus a recent commercial layer that does, frankly, a different job from either. I worked through each of these as part of writing this paper, ran the one engine I could run end-to-end against the chardet v6/v7 pair, and report what came back.

Token-based academic detectors (run configuration). The JPlag empirical contrast that motivates this paper appears earlier; see Section 1.1 for the headline numbers, the comparison table (Table 1), and the argument that token-level matching cannot see what call-graph and control-flow signals can. Here we record only the run configuration and the reproduction artefacts that did not fit the introduction. JPlag v6.3.0 was run on the chardet v6.0.0 and v7.0.0 pair with default minimum token match (12) under its bundled Python 3 grammar. The reproduction script is in `chardet-relicense/manuscript/figures/scripts/run_jplag.sh`, the raw JSON output in `chardet-relicense/manuscript/figures/scripts/jplag_chardet_results.json`.

AST-based academic detectors. A separate research lineage compares programs at the AST level rather than the token-string level. CodEx [35] is a representative recent design and worth a closer look: it is a source-code search engine (you query a snippet against a corpus of snippets) that combines ASTs with entity division, node weights, hash values, sibling sort (to defeat statement reordering), and a weighted depth-first similarity match. The authors report 95% of test cases identified even when common detection-avoidance techniques are applied, which is a strong claim and one I would want to see independently replicated before treating it as a hard number. Like AST-CC and the earlier clustering-based AST entries surveyed by Roy et al. [22] and Moussiades and Vakali [19], CodEx is best understood as a research artefact rather than a packaged tool: there is, frankly, no canonical binary you can download and run against the chardet pair, and reimplementing it end-to-end was outside the scope of this paper. I cite CodEx as the correct intellectual neighbour

for the C06a and C06c signals, and I would name AST-CC as an algorithm-in-the-literature rather than a tool, which matches how it appears in survey papers.

Commercial AI-augmented tools. The third strand is a set of recent commercial products that advertise AI-augmented detection, and this is where I want to push back gently on the marketing. *Copyleaks Code (Codeleaks)* [6] is a real, database-backed commercial code checker that claims cross-language paraphrase detection across 100+ languages, plus AI-generated-code flagging. Independent reviews, to be fair to the vendor, are reasonably consistent that accuracy degrades on paraphrased or "humanised" content, and that is an industry-wide limit rather than something specific to Copyleaks. I could not run Copyleaks against the v6/v7 pair as part of writing this paper because the product is SaaS-only and requires an account; the published interface (upload to vendor servers) is also a posture-incompatible model for a paper whose entire methodology requires every reviewer to re-derive every number locally.

CodeSpy AI [5] is doing something narrower than the others, and the distinction matters more than the marketing suggests. It is an AI-vs-human classifier (so it estimates the probability that a given source file was generated by a model like ChatGPT, Gemini, or Claude, across roughly six languages: Java, Python, JavaScript, C++, C#, PHP) and it outputs a confidence score rather than a binary verdict. It is not a similarity or collusion detector. Detecting "was this written by a model" is, to be precise, a strictly different statistical task from detecting "does this share structure with that." The cross-vendor evidence on AI-vs-human classifiers is that they are accurate on raw LLM output and degrade sharply on human-edited or humanised LLM output, which is roughly what the underlying statistics predicts. I treat vendor-published accuracy figures (the 98–99% range that turns up on lab corpora) as best-case numbers, and assume real-world false-positive and evasion rates are meaningfully higher. The C06 signals are similarity signals against a known prior, full stop; they do not assert anything about the generative process of either v6 or v7.

Where this paper fits. The C06 signal set is methodologically closest to the AST-based academic lineage (CodEx, AST-CC, GumTree). It does not try, and should not be expected, to occupy the JPlag niche (cohort similarity at the token-string level) or the Copyleaks/CodeSpy niche (paraphrase- or AI-generation-classification at the file level). What this paper adds beyond the academic AST detectors is two things: a small fixed substrate (Section 2.4) so the contracts under which a similarity number was produced are themselves machine-readable and verifiable, and a worked application against a real, disputed pair where the answer matters.

10.3 LLM code memorisation

The empirical record on LLM memorisation of training-set code is short but consistent. Ziegler reported on Copilot's *Recitation Rate* — the rate at which long outputs match training-set strings verbatim — and observed that such recitation is rare but non-zero [36]. Carlini et al. showed that memorisation in neural language models is a strictly increasing function of model scale, training repetition, and prompt context length, with no known saturation point [4]. Both pieces of evidence are load-bearing for the FSF director's position quoted in Section 2.1 [28]: an LLM that ingested chardet v6 during pretraining cannot be assumed to have forgotten it. The detection harness in this paper does not measure training-data provenance and cannot resolve the memorisation question; what it does is shrink the space within which the dispute can be conducted by producing output-only structural signals that a reviewer can re-derive from the artefacts a court would actually see.

11 Limitations and Future Work

The harness's reference extractor is Python-specific, although the v2 revision restructures the extractor around a language-neutral walker protocol; see Section 11.1 for the seam. The DAG-TOML scaffolding has always been language-neutral; the v2 revision brings the extractor to the same standard structurally, even though only the Python reference walker is shipped.

The v1 paper observed that C06a is module-level and that a surgical per-function paraphrase preserving the module-level call graph could go undetected. The v2 revision closes that gap with C06f (Section 4.8), which reports a per-function match rate and a within-match similarity score; the v6/v7 result (Section 6.3) is the worked instance. A further refinement — per-method signature equivalence on classes — is also folded into the v2 C06d (per-method walker, Table 6).

The C06d signal does not adjudicate when the two `__all__` class sets are fundamentally different; the v6/charset_normalizer pair is the worked degeneracy (Section 4.6). When public-API shapes are disjoint the calibration evidence has to come from elsewhere in the suite.

C06e requires the test runner to be able to `pip install` both versions from local worktrees. Where that fails (missing native toolchain, missing system libraries), the signal emits `SKIP`, not `FAIL`, and the gate distinguishes the two. A future revision could ship pre-built wheels of both versions and avoid the `pip install` step entirely.

The behavioural fingerprint's v1 random-bytes workload is now complemented by a realistic corpus (HTML, multilingual text, RFC bodies, email; $n = 64$) introduced in v2 in response to reviewer concern R3 that random bytes are the wrong workload for an encoding detector. A larger realistic corpus and a targeted input fuzzer that produces inputs known to exercise specific code paths in both versions would each be strictly stronger; both remain future work.

The harness addresses output similarity. It does not address training data provenance. A future signal that joined the harness's output to a sufficiently large model-training-data audit — which the deployment context for closed models does not currently permit — would address the question the FSF position raises [28].

C06f matcher dependence. C06f's discriminative power depends on the `(signature_shape, fan_in_bucket, fan_out_bucket)` matching key. The signature-shape hash incorporates an annotation-count dimension, which is dominated by v7's systematic addition of type hints — functions that v7 annotated and v6 did not are mechanically unmatched. A signature-blind or annotation-blind matcher would yield different match rates: an ablation that collapses every function into a single coarse shape bucket brings v6/v7 and v6/charset_normalizer to similar within-match similarities (in the 0.89–0.90 band) and removes the cross-pair separation reported in Table 6. We report the specific matcher's result here and flag the design choice as an open ablation; the matcher key, including the annotation-count dimension, is the load-bearing assumption behind the v6/v7 "low match rate, high within-match similarity" framing.

Training-data contamination of the prior. If the LLM coding agent that produced v7 had encountered v6 or chardet-derived material during pre-training, every structural-similarity signal in this paper is potentially confounded by memorisation rather than informed by independent derivation or convergent design. The harness cannot measure training-data exposure; the most we can do is to flag the risk and to interpret the calibration baselines (v5/v6, v6/charset_normalizer) in light of it. v6/charset_normalizer in particular may not be a true "independent" baseline if the same training corpus saw both libraries. We treat this as a current threat to validity of the v6/v7

interpretation, not only as future work.

Realistic-corpus independence. The 64-item realistic corpus contains pairs of items that are byte-identical across encodings that the harness's label-aliasing rule treats as equivalent (e.g. the UDHR Article 1 English text is byte-identical when serialised as ISO-8859-1 vs Windows-1252, and the aliasing rule treats those labels as a family match). The 64 rows are therefore not 64 independent observations; the normalised-rate band of 0.594–0.688 may be partly shaped by the corpus's encoding-equivalence structure. A deduplicated-by-SHA sensitivity analysis is future work.

11.1 Toward language-neutral extraction

The structural-similarity signals C06a, C06a′, C06b, C06c, C06d and C06f all consume one specific view of the implementation under test (a call graph, an import edge set, a control-flow node histogram, a public-API signature surface, or a per-function shape record). In the v1 paper these views were extracted directly via Python's `ast` module, hard-coding the implementation language into the signal definitions even though the signal *contracts* are language-agnostic (a call graph is a call graph in any language). Phase 1b of the v2 revision restructures the extractor around an `ASTWalker` protocol (Figure 4): each signal calls only `walker.iter_call_edges()`, `walker.iter_control_flow_nodes()`, `walker.iter_public_api()`, `walker.iter_imports()`, `walker.iter_class_methods()`, or `walker.iter_function_signatures()`, with no signal touching an `ast.*` type or any other language-specific parser API directly. The reference instance, `PythonASTWalker`, satisfies the protocol via the existing Python `ast` traversals (each protocol method maps to a well-defined set of AST node classes; the diagram annotates the mapping). Two hypothetical sibling instances, `TreeSitterRustWalker` (Rust via `syn` or tree-sitter) and `GoASTWalker` (Go via `go/ast`), are shown as un-implemented stubs alongside the Python walker; each protocol method is annotated with the language-native construct (`syn::ExprCall`, `go/ast.CallExpr`, etc.) a plugin author would need to wire up. We do **not** ship Rust or Go implementations; the claim being verified is structural, not empirical: that the signal definitions, as a body of code, no longer entangle the choice of implementation language with the choice of similarity contract. The refactor preserves every numeric output byte-for-byte against the pre-refactor harness — the same point estimates, the same percentile-bootstrap CIs, and the same per-method verdict counts — so it is a purely structural intervention. Reviewer concern R15 (the v1 "language-neutral spec" claim was unverified in code) is thereby answered by exhibiting the seam; the v1 prose is preserved unchanged and is now substantiated by the code structure.

12 Conclusion

The `chardet` 7.0 dispute is the first widely-discussed test of whether the clean-room reverse-engineering doctrine extends to AI-driven rewrites, and I expect it will not be the last. The answer to whether v7's relicensing is valid is not in the bundle I ship and is not in this paper; the answer requires a court, a legal theory, and the relevant evidence (and on the legal theory the paper is deliberately silent, because that is not my expertise to render). What the bundle ships is the evidence: six paraphrase-resistant, reproducible numbers any reviewer can re-derive from the cited source artefacts, every one of which has been independently cross-validated against a second reference implementation in `chardet-relicense/manuscript/figures/scripts/validate_numbers.py` (AUX1 and C06a-d via scipy / numpy second-source primitives; C06e via a deterministic stdlib digest re-derivation plus a subprocess to the harness's behavioural-fingerprint script that recovers the exact-match and bucket-match rates, with explicit SKIP semantics when the runner cannot install chardet to

re-execute it). On the actual disputed release pair, the numbers describe a rewrite whose internal structure is highly preserved relative to v6, whose third-party dependency boundary is genuinely different, whose public API is mostly preserved by signature, and whose runtime behaviour is operationally distinct. The combined picture is internally consistent and legally ambiguous: v7 preserves the shape of v6's thinking while replacing what v6 actually does. The right next step is for legal and technical reviewers to re-run the harness, agree on what the numbers mean, and disagree about the rest in public. The wider purpose, beyond chardet specifically, is to demonstrate that AI-driven software decisions can be made auditable in a form courts and regulators can independently re-derive.

Acknowledgments

Human contributor. Werner Kasselman (Verivus OSS, `werner@verivus.com`) provided the direction, scoping, and judgment calls throughout: the initial framing of the problem ("write a proof that shows the spec applied to code"), the narrowing of scope to `chardet` specifically (against an initial agent recommendation that included another LLM-driven rewrite incident and the Quake III Copilot incident), the push-back when the v0.1 of the proof tested only file hashes ("I don't know how this really tests the spec, semantic ast framing"), the decision to do the full rewrite to AST-level signals rather than incremental fixes, the iteration cadence rules for the multi-LLM review (each reviewer dispatched with full filesystem access and the same MCP toolset; permission grants auto-issued; on each iteration, if a permission grant did not persist durably, progress polled no more often than once every 90 seconds), the rejection criteria ("approval must be based on inspected code, tests, docs, and persistent review evidence; do not ask reviewers to approve based on intent, plan compliance claims, or 'should be fixed' language"), and the requirement that this acknowledgments section be explicit about who did what. The broader DAG-TOML specification that the proof bundle plugs into was likewise scoped, edited, and version-controlled by Kasselman over many prior sessions. The human did not author the LaTeX or the Python; the human authored the questions, the scoping, and the acceptance criteria.

LLM authoring. The manuscript was drafted with the assistance of an LLM coding agent, working from the bundle commit at HEAD `6acef08` in the `verivus-oss/agent-assurance-papers` repository. The proof bundle itself was authored over several sessions of the same agent harness.

LLM review. Three independent LLM reviewers verified the bundle and the paper across multiple iterations using a shared `contract-declaration` "verification report" as the corrective-program spec (Section 9). The first reviewer (`workspace-write` sandbox, `on-request` approvals) iterated three rounds on the bundle and two rounds on the paper, reaching UNCONDITIONAL APPROVAL on all 15 bundle contracts and on the manuscript after all blockers were addressed. The second reviewer (`bypassPermissions`, `alwaysApprove`) reached UNCONDITIONAL APPROVAL on all 15 bundle contracts in one round and on all 20 paper contracts after a second round. The third reviewer (`yolo` access posture) reached UNCONDITIONAL APPROVAL on the first-round bundle review; its second-round bundle review and both paper-review rounds failed before completion with an upstream `TerminalQuotaError`, so the third reviewer's verdict on the final paper version is not part of the record at submission time. We note the gap explicitly rather than silently omitting it. All three reviewers shared the same MCP set (`sqry`, `ref_tools`, `exa`); the itemised list of issues each reviewer caught is preserved in the v2 review packet at `chardet-relicense/manuscript/v2-phase1a-review-packet.md` and `chardet-relicense/manuscript/`

`v2-phase1a-review-round2-responses.md` in the same repository.

The multi-LLM review is a methodological note, not an external peer review and not a guarantee of correctness. Disclosing it is not a disclaimer of responsibility: the Verivus OSS organisation and the named human contributor accept responsibility for what the paper claims. It is a disclosure of process, because process matters when the subject is process.

Verbatim user-prompt record. The verbatim user prompts that produced both the proof bundle and this manuscript are recorded at `chardet-relicense/manuscript/user-prompts.md` in the same repository. The file is referenced from the body of the manuscript rather than reproduced verbatim here because the prompts are a process artefact, not a research result; readers who want to trace any implementation or authoring decision back to the human directive that motivated it can do so via that file.

References

[1] Ars Technica. AI can rewrite open-source code – but can it rewrite the license too?, March 2026.

[2] Ira D. Baxter, Andrew Yahin, Leonardo Moura, Marcelo Sant'Anna, and Lorraine Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (ICSM '98)*, pages 368–377, Bethesda, MD, USA, 1998. IEEE Computer Society.

[3] Cristian Cadar, Daniel Dunbar, and Dawson Engler. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI '08)*, pages 209–224. USENIX Association, 2008.

[4] Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramèr, and Chiyuan Zhang. Quantifying memorization across neural language models. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023.

[5] CodeSpy AI. CodeSpy AI: Ai vs human source-code classifier. Commercial product page, 2026. Classifier (not a similarity tool); estimates the probability that source code was generated by a large language model. Six languages supported at product launch (Java, Python, JavaScript, C++, C#, PHP). Outputs a confidence score, not a binary verdict.

[6] Copyleaks. Codeleaks: AI-powered source code plagiarism detection. Commercial SaaS product page, 2026. Database-backed; vendor claims cross-language detection and paraphrased-code flagging across 100+ languages; independent reviews report accuracy degrades on humanised paraphrase.

[7] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*, pages 313–324. ACM, 2014.

[8] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model

for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.

[9] @gooba42. Nullified license (issue #325). GitHub issue, chardet/chardet repository. Confirmed via GitHub REST API: `api.github.com/repos/chardet/chardet/issues/325` reports `user.login` = `gooba42` and `title` = "Nullified License"., March 2026.

[10] John Gruber. Can coding agents relicense open source through a "clean room" implementation of code? Daring Fireball, March 2026.

[11] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. Proceedings of the 7th Python in Science Conference (SciPy 2008); ongoing project at networkx.org, 2008.

[12] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In Gaël Varoquaux, Travis Vaught, and Jarrod Millman, editors, *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, pages 11–15, Pasadena, CA, 2008.

[13] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[14] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondu. DECKARD: Scalable and accurate tree-based detection of code clones. In *Proceedings of the 29th International Conference on Software Engineering (ICSE '07)*, pages 96–105. IEEE Computer Society, 2007.

[15] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. Soufflé: On synthesis of program analyzers. In *Proceedings of the 28th International Conference on Computer Aided Verification (CAV '16)*. Springer, 2016.

[16] JPlag contributors. JPlag — detecting software plagiarism. release v6.3.0. GitHub repository, Karlsruhe Institute of Technology, 2026. Release used for the empirical comparison reported in Section 9.4. Token-string engine (RKR-GST); local-only execution; no upload of submissions to any external service.

[17] LWN.net. The relicensing of chardet, 2026.

[18] Heather Meeker. The chardet controversy: open source and the AI clean room. Copyleft Currents, April 2026.

[19] Lefteris Moussiades and Athena Vakali. PDetect: A clustering approach for detecting plagiarism in source code datasets. *The Computer Journal*, 48(6):651–661, 2005. Cited in surveys as an early entry in the AST-based clone-detection family alongside AST-CC; we cite AST-CC by its survey appearances rather than a single canonical paper.

[20] Mark Pilgrim. No right to relicense this project (issue #327). GitHub issue, chardet/chardet repository, March 2026.

[21] Lutz Prechelt, Guido Malpohl, and Michael Philippsen. Finding plagiarisms among a set of programs with JPlag. *Journal of Universal Computer Science*, 8(11):1016–1038, 2002. RKR-GST token-string similarity engine; later releases described at the v6 repository, https://github.com/jplag/JPlag, urldate 2026-05-22.

[22] Chanchal K. Roy, James R. Cordy, and Rainer Koschke. Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming*, 74(7):470–495, 2009.

[23] Shuji Sado. Can you relicense open source by rewriting it with AI? the chardet 7.0 dispute. Open Source Guy (shujisado.org), March 2026.

[24] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research*, 12:2539–2561, 2011.

[25] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Grosen, Siji Feng, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. SOK: (state of) the art of war: offensive techniques in binary analysis. In *Proceedings of the 2016 IEEE Symposium on Security and Privacy (S&P '16)*, pages 138–157. IEEE, 2016. Introduces the angr binary-analysis framework.

[26] Supreme Court of the United States. Baker v. selden, 101 u.s. 99 (1880). U.S. Reports, 1880.

[27] Supreme Court of the United States. Google llc v. oracle america, inc., 593 u.s. _____, 141 s. ct. 1183 (2021). Slip opinion No. 18-956, 2021.

[28] The Register. AI kills software licensing: chardet rewrite stirs open-source row, March 2026.

[29] United States Court of Appeals, Ninth Circuit. Sega enterprises ltd. v. accolade, inc., 977 f.2d 1510 (9th cir. 1992). Federal Reporter, Second Series, 1992.

[30] United States District Court, N.D. California. Phoenix technologies, ltd. v. ibm corp. (clean-room BIOS reimplementation, 1984). Industry summary; see Sega v. Accolade for legal reception of the technique, 1984.

[31] Verivus OSS. DAG-TOML specification (release candidate 1.0.0) and agent assurance profile. GitHub: verivus-oss/agent-assurance, 2026.

[32] Verivus OSS. sqry: AST-based semantic code search via a unified symbol-and-relationship graph. GitHub: verivus-oss/sqry; `crates.io`: `sqry-cli`, 2026. Used in this paper to independently corroborate C06a's call-graph topology numbers via the MCP tools `get_graph_stats`, `find_cycles`, `get_insights`, `direct_callers`, `direct_callees`, and `find_duplicates`.

[33] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[34] Simon Willison. Can coding agents relicense open source through a clean-room implementation of code? Personal weblog, March 2026.

[35] Wei Zheng, Qiongqiong Pan, and David Lillis. CodEx: Source code plagiarism detection based on abstract syntax tree. In *Proceedings of the 26th Irish Conference on Artificial Intelligence and Cognitive Science (AICS)*, 2018. Pure-AST source code search engine with entity division, node weights, hash values, sibling sort, weighted DFS match.

[36] Albert Ziegler. GitHub Copilot research recitation. The GitHub Blog, June 2021.

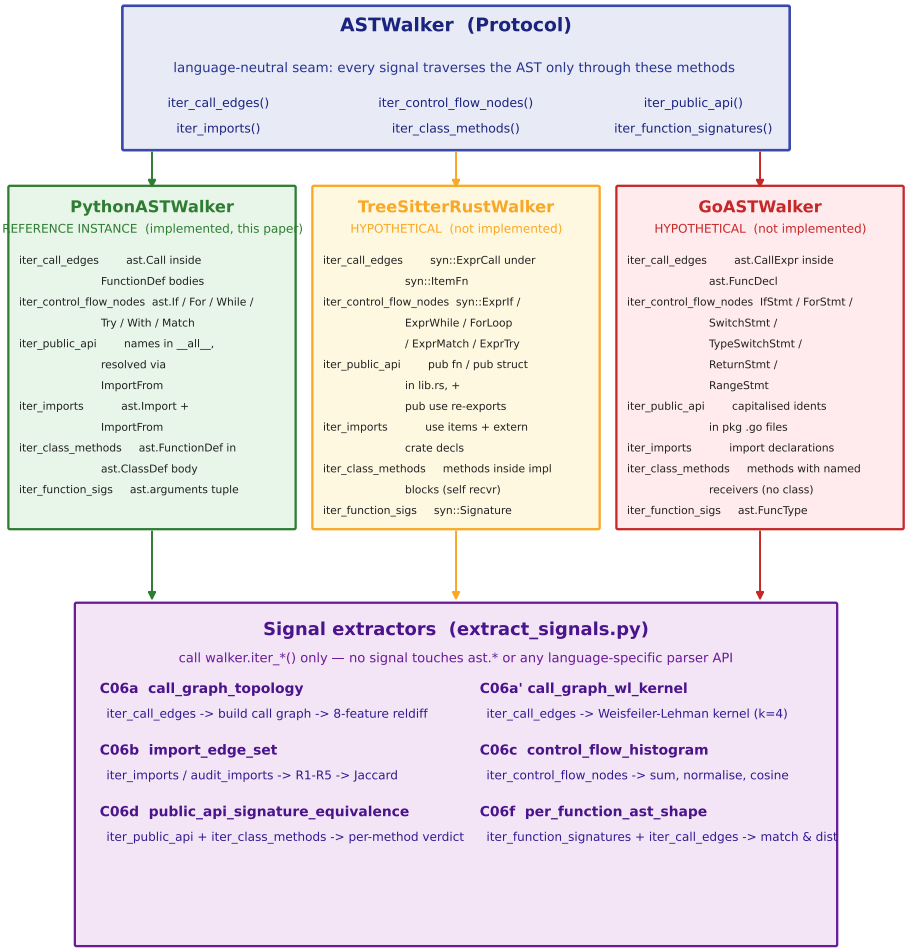

Figure 5. The language-neutral AST-walker seam introduced in v2 Phase 1b.
Python is the reference instance; Rust and Go instances are stubs documenting the equivalent constructs.

Figure 4: The v2 `ASTWalker` protocol seam. Each structural-similarity signal calls only the protocol's six language-neutral iterator methods; the reference `PythonASTWalker` satisfies the protocol via the existing Python `ast` traversals, and two hypothetical sibling instances (`TreeSitterRustWalker`, `GoASTWalker`) are shown with the language-native construct each protocol method would map to. The Rust and Go walkers are illustrative stubs only — the v2 paper ships the Python reference walker.